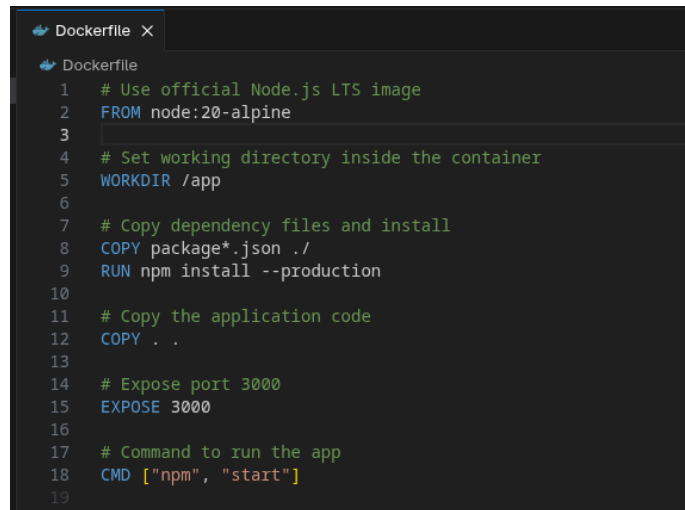


DevOps – Program – 03

Title: Code a Dockerized Python Flask or Node.js Application



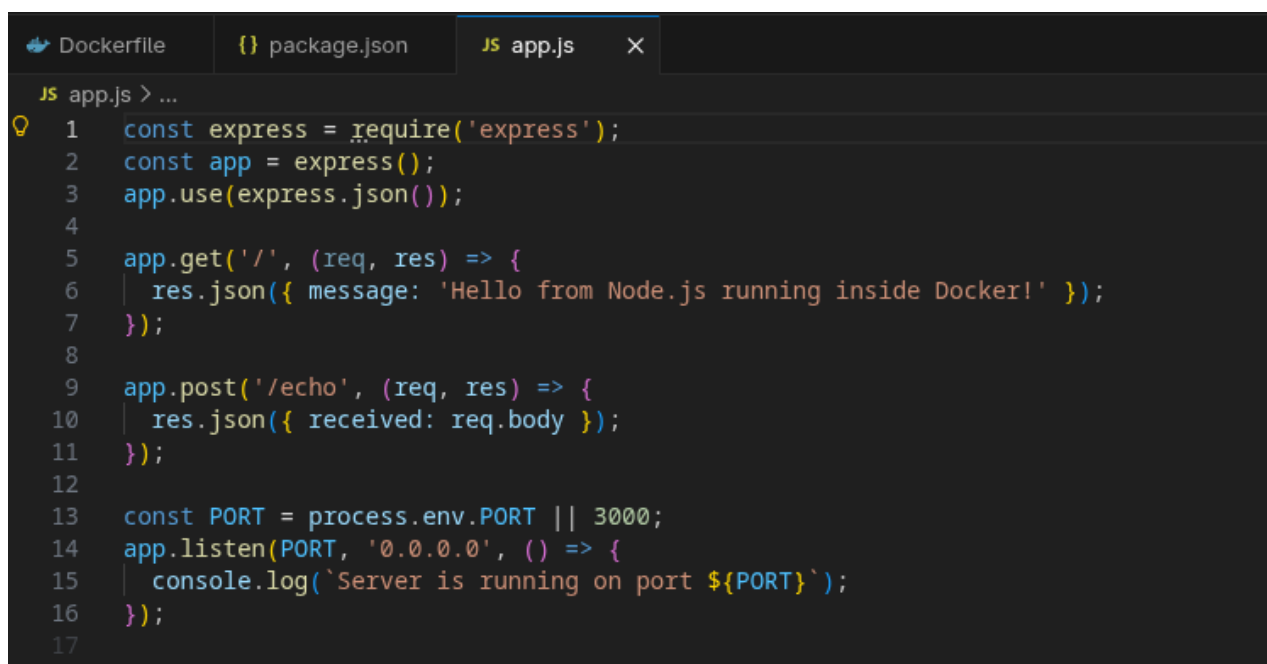
```
Dockerfile X
Dockerfile
1 # Use official Node.js LTS image
2 FROM node:20-alpine
3
4 # Set working directory inside the container
5 WORKDIR /app
6
7 # Copy dependency files and install
8 COPY package*.json ./
9 RUN npm install --production
10
11 # Copy the application code
12 COPY . .
13
14 # Expose port 3000
15 EXPOSE 3000
16
17 # Command to run the app
18 CMD ["npm", "start"]
19
```

This image illustrates the **Dockerfile** used to containerize the Node.js application. The **Dockerfile** starts by using the official **Node.js 20 Alpine** base image, which provides a lightweight, efficient, and stable runtime environment.

To improve documentation and traceability, metadata such as the maintainer's name, application version, and build date are included. The container's working directory is set to /app, ensuring that all application-related operations occur within this directory.

The **package.json** file is copied first, allowing the required production dependencies to be installed using **npm install**. After the dependencies are successfully installed, the remaining application source code is copied into the container. Port 3000 is exposed to enable external access to the Node.js application.

Finally, the instruction **CMD ["npm", "start"]** defines the command that is executed when the container starts. Overall, this Dockerfile serves as the foundation for building the container image.



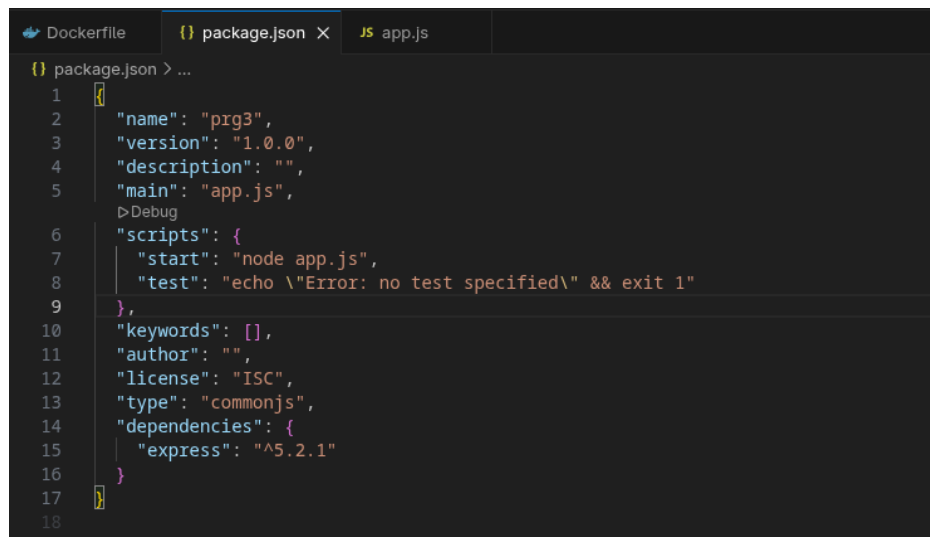
```
Dockerfile {} package.json JS app.js X
JS app.js > ...
1 const express = require('express');
2 const app = express();
3 app.use(express.json());
4
5 app.get('/', (req, res) => {
6   res.json({ message: 'Hello from Node.js running inside Docker!' });
7 });
8
9 app.post('/echo', (req, res) => {
10   res.json({ received: req.body });
11 });
12
13 const PORT = process.env.PORT || 3000;
14 app.listen(PORT, '0.0.0.0', () => {
15   console.log(`Server is running on port ${PORT}`);
16 });
17
```

This screenshot depicts the implementation of a simple Express.js server defined in the app.js file. The code represents a minimal Node.js web application designed to run inside a Docker container. It imports the Express framework and initializes an application instance to handle incoming HTTP requests.

The application defines two HTTP endpoints:

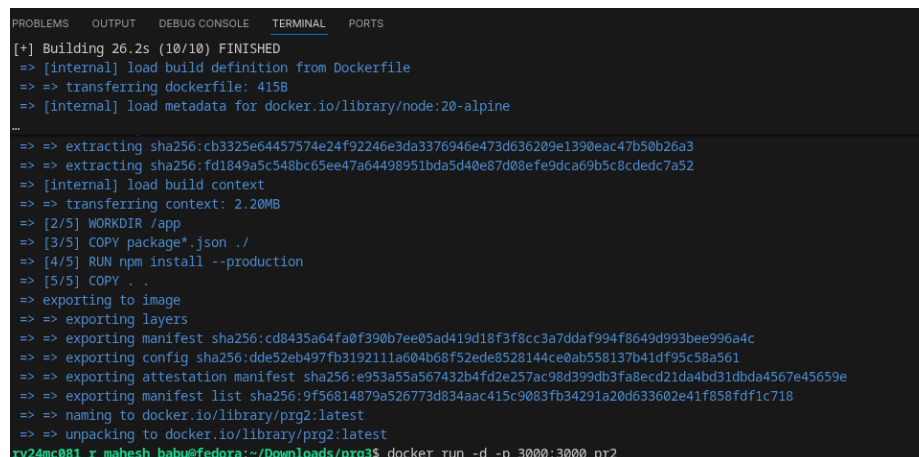
- **GET /** – Sends a JSON response indicating that the Node.js application is successfully running inside a Docker container.
- **POST /echo** – Accepts data from the client request and returns the same data back in JSON format, demonstrating request handling and response processing.

The server is configured to listen on port 3000 by default or on a custom port specified through the environment variable PORT. Upon successful startup, a console message displaying “Server is running on port \${PORT}” confirms that the application is active.



```
Dockerfile package.json X JS app.js
package.json > ...
1 {
2   "name": "prg3",
3   "version": "1.0.0",
4   "description": "",
5   "main": "app.js",
6   "scripts": {
7     "start": "node app.js",
8     "test": "echo \"Error: no test specified\" && exit 1"
9   },
10  "keywords": [],
11  "author": "",
12  "license": "ISC",
13  "type": "commonjs",
14  "dependencies": {
15    "express": "^5.2.1"
16  }
17
18
```

This image displays the package.json file, which defines the project metadata and dependencies for the Node.js application. It includes essential information such as the project name (program3), version, main entry point (app.js), and startup command (node app.js). The "dependencies" section lists Express.js (version ^5.1.0) as the primary framework used to build and run the web server. The "scripts" section specifies the "start" command that is executed when the container runs the application. This file plays a crucial role in dependency management and automation of execution commands, ensuring that all required packages are installed and the application can be started seamlessly within the Docker environment.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[+] Building 26.2s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> transferring dockerfile: 415B
=> [internal] load metadata for docker.io/library/node:20-alpine
...
=> extracting sha256:cb3325e64457574e24f92246e3da3376946e473d636209e1390eac47b50b26a3
=> extracting sha256:fd1849a5c548bc65ee47a64498951bda5d40e87d08ef9dca69b5c8cdedc7a52
=> [internal] load build context
=> transferring context: 2.20MB
=> [2/5] WORKDIR /app
=> [3/5] COPY package*.json ./
=> [4/5] RUN npm install --production
=> [5/5] COPY . .
=> exporting to image
=> exporting layers
=> exporting manifest sha256:cd8435a64fa0f390b7ee05ad419d18f3f8cc3a7ddaf994f8649d993bee996a4c
=> exporting config sha256:dde52eb497fb3192111a604b68f52ede8528144ce0ab558137b41df95c58a561
=> exporting attestation manifest sha256:e953a55a567432b4fd2e257ac98d399db3fa8ecd21da4bd31dbda4567e45659e
=> exporting manifest list sha256:9f56814879a526773d83aac415c9083fb34291a20d633602e41f858df1c718
=> naming to docker.io/library/prg2:latest
=> unpacking to docker.io/library/prg2:latest
rv24mc081_r_mahesh_babu@fedora:~/Downloads/prg3$ docker run -d -p 3000:3000 pr2
```

This section illustrates the steps involved in building a Docker image and running a container based on that image.

Preliminary Commands:

The process begins by navigating to the project directory using `cd Program3` and verifying the required files with the `ls` command.

Docker Image Build:

Command: `docker build -t pr2-img .`

Purpose: Builds a Docker image using the Dockerfile located in the current directory and assigns it the tag `pr2-img`. The build output confirms a successful multi-step (11-step) image creation process, with several steps utilizing cached layers to optimize build time.

Running the Docker Container:

Command: `docker run -d -p 3000:3000 pr2-img`

Purpose: Launches a container from the `pr2-img` image.

`-d` runs the container in detached (background) mode.

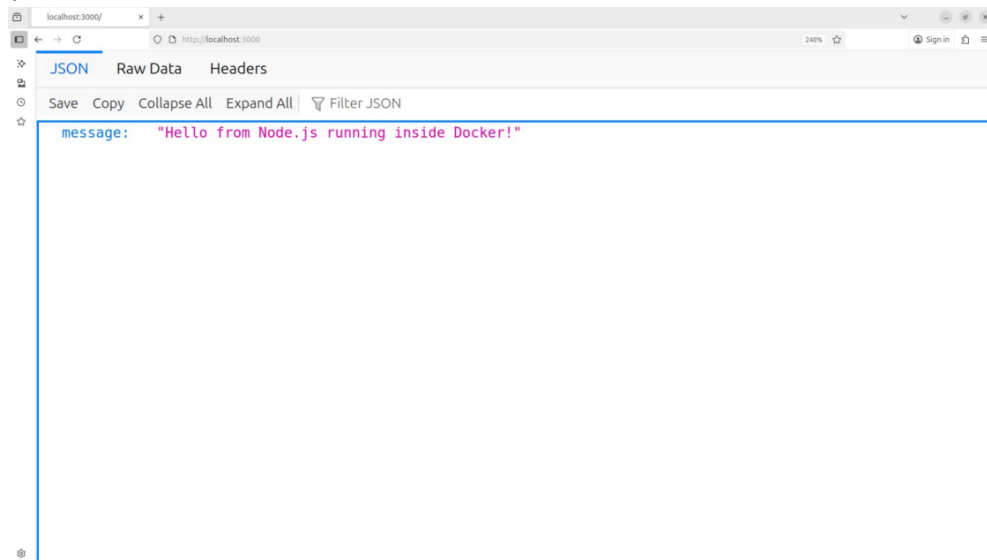
`-p 3000:3000` maps port 3000 of the container to port 3000 on the host system, allowing external access to the application.

The command outputs a unique container ID (for example, `1b30460...`) indicating successful container startup.

Viewing Container Logs:

Command: `docker logs <container_id_prefix>`

Purpose: Displays the runtime logs of the running container, helping verify that the application has started correctly and is functioning as expected.



This shows the application being successfully accessed via a web browser.

- **URL:** `http://localhost:3000/`
- **Result:** The browser displays the JSON response from the Node.js application's root endpoint (`/`).
- **Content:** `{"message": "Hello from Node.js running inside Docker!"}`
- **Verification:** This confirms the entire Dockerization process was successful: the image was built, the container is running, the port forwarding is working, and the application is responding correctly.